MINI PROJECT | ICT 2403 | GRAPHICS AND IMAGE PROCESSING

Rajarata University of Sri Lanka, Department of Computing

Monitoring Plant Growth in Controlled Environments Using Image Processing

FINAL REPORT

https://github.com/RashmikaKDH/Imageprocessing-group-project-PlantGrowthMonitor

GROUP UNIFY

W.K.D.Bhagya	WB293	ICT/2022/110	5713
M.T.Rathnayake	MR1766	ICT/2022/114	5716
R.D.Malsha Nethmini	MN1032	ICT/2022/085	5689
K.D.Heshan Rashmika	KR1211	ICT/2022/103	5706

Table of Contents

1. Introduction

1.1. Problem:

Create and implement an image processing system that can track the development of plants over time under regulated settings, such a greenhouse. Analyze plant health indicators, leaf count, and growth trends over time using image processing methods.

1.2. Importance of using image processing to monitor plant growth.

1.2.1. Automated Monitoring and Control:

Image processing can be integrated with automated systems for continuous monitoring and control of plant growth environments.
This enables precise adjustments to irrigation, fertilization, and other factors, optimizing plant growth and resource utilization.

1.2.2. Early Detection of Stress and Disease:

Image processing can detect subtle changes in plant appearance that may indicate stress or disease before they become visible to the human eye.
Early detection allows for timely intervention, minimizing crop losses and improving plant health

2. Image Acquisition Model

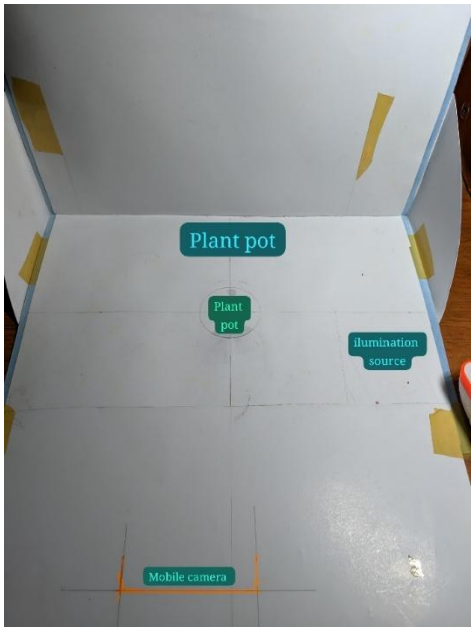
2.1.Objective

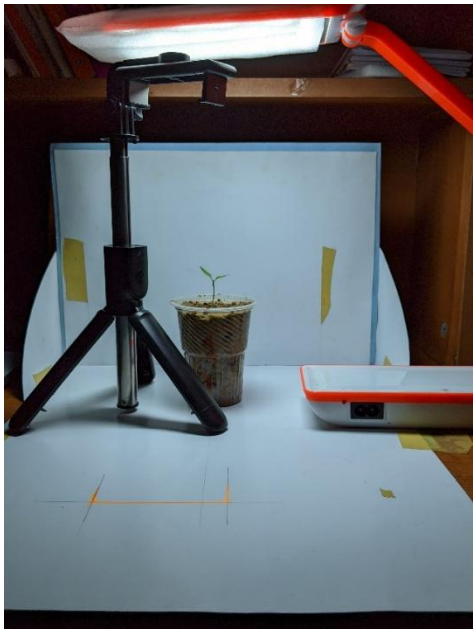
The goal is to create an image acquisition model for chili plants, capturing detailed, accurate images for growth monitoring, identifying health indicators, integrating into automated systems, and ensuring reliability.

2.2.Experimental Setup

A room was chosen for control, small pots were used for plant movement, a solid white background was used for contrast, and a tripod was used to keep the camera in the same position and angle.

The equipment includes tripods and LED light for camera mounting. Images are taken three times a day, with 12 photos per day. The resolution is 12.2MP, with a frame rate of 1/20 sec. Data is stored on the computer, with a flash drive as an external storage. JPG files are used due to their smaller size.





2.3.Challenges and Limitations:

- To take better photos at night, we had to use artificial lighting.
- We can't get a hundred percent healthy plant because nutrition and water supply are not the same.
- Effects happen on climate change to plants.

3. Pre-processing methods

When dealing with distortions in an image set—like salt-and-pepper noise, non-uniform illumination, and blur—we need a systematic pipeline to restore the images effectively. Here's the we created order and why:

3.1. Correct Non-Uniform Illumination

- Why first? Uneven lighting affects the contrast and intensity distribution, which can make later noise and blur removal less effective. Applying histogram equalization, adaptive contrast enhancement, or homomorphic filtering first ensures that noise and edges are more uniformly distributed.
- Correct Non-Uniform Illumination – Since we used different light sources (daylight and artificial light), there may be intensity variations. Applying adaptive histogram equalization or homomorphic filtering will help normalize brightness across frames

3.2. Remove Salt-and-Pepper Noise (Impulse Noise)

- Why second? If we apply blur reduction first, the noise can spread and become harder to remove. Salt-and-pepper noise is best handled using a median filter or non-local means filtering before sharpening or deblurring the image.
- Remove Salt-and-Pepper Noise – If any small pixel artifacts appear due to sensor noise, a median filter can help clean them up.

3.3. Remove Gaussian or Speckle Noise (if present)

- Why here? After impulse noise is gone, remaining Gaussian or speckle noise (often present in low-light conditions or sensor defects) can be removed using techniques like bilateral filtering, wavelet thresholding, or deep-learning-based denoising.
- Reduce Gaussian Noise (if needed) – If our alternative light source introduced grainy noise, use a bilateral filter or a deep-learning-based denoiser.

3.4. Final Optional Step: Edge Sharpening or Super-Resolution (if needed)

- Why last? This step should be applied only when the image is free of distortions to enhance details without amplifying noise or artifacts.
- Final Enhancement – If needed, apply sharpening to enhance plant details.

1_reducing_fixing_non-uniform illumination_CLAHE_YCrCb

2_removing salt-and-pepper noise_median_filtered

3_Remove Gaussian noise_bilateral_filter

Original image

4_sharpened_improved

In this sharpened improved image, non-uniform illumination has been reduced, salt & pepper noise and gaussian noise have been removed, and the image has become smooth without edge blur.

4. Image Segmentation Model Report

4.1. Introduction

Image segmentation is a crucial step in monitoring plant growth, as it enables the isolation of plant regions from background elements in captured images. This process helps in analyzing growth parameters such as leaf count, structural changes, and health indicators over time. The primary objective of this segmentation model is to develop an efficient algorithm that accurately detects and separates plant regions from their surroundings, ensuring reliable analysis.

4.2. Segmentation Setup

4.2.1. Software and Tools Used

- Programming Language: Python
- Libraries: OpenCV, Matplotlib
- Environment: Jupyter Notebook

4.2.2. Segmentation Techniques Applied

4.2.2.1. Color-based Segmentation

- Uses the HSV (Hue, Saturation, Value) color space to threshold based on color. Typically targets specific colors in the image..

Method	Description	Type	Use Case	Strengths	Weaknesses
Color-Based HSV Thresholding	Uses the HSV (Hue, Saturation, Value) color space to threshold based on color. Typically targets specific colors in the image.	Color-Based, Binary	Best for segmenting images based on specific colors (e.g., detecting red, blue, green objects).	Effective for color-based segmentation, independent of lighting conditions.	Sensitive to color variations (lighting can change perceived color), and works best with distinct colors.

Each thresholding method has its ideal scenarios depending on the nature of the image (e.g., lighting, color distribution, and complexity).Because we are working with color image we choose color based segmentation to separate plant from the background.

4.3.Segmentation Parameters and Consistency

- Define HSV range for green color using colorsegmanter which empowers users to dynamically isolate specific color ranges within an image
- Create mask for green areas (plant)
- Apply morphological operations to clean the mask.
- Apply the mask to the original image to isolate green areas(plant).

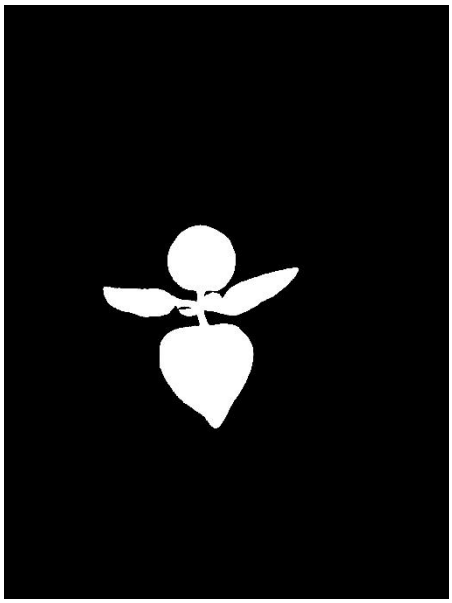
4.4.Segmentation Discussion

segment plant regions from their background

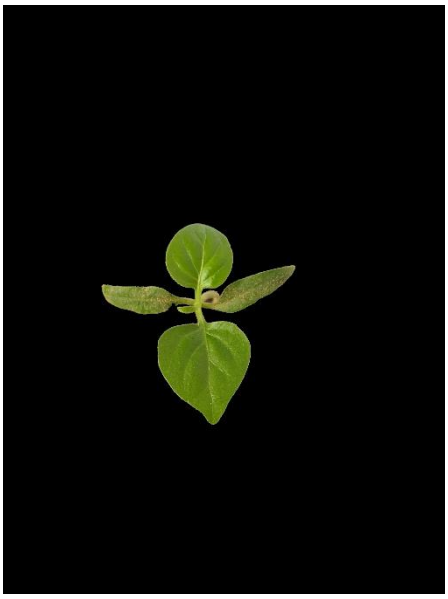
Steps in the Algorithm:

- Load Image: Reads the plant image and converts it to RGB.
- Convert to HSV: HSV is better for color-based segmentation.
- Create Mask: Filters out green plant regions.
- Morphological Processing: Cleans noise using opening and closing operations.

Original image



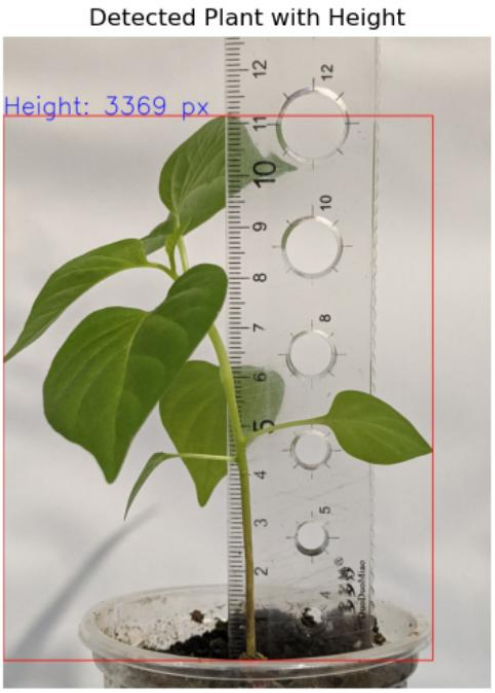
Green Mask



Segmented plant

5. Plant height detection

We can measure the height of a plant, but to get it in a unit like centimeters, we need to get a ratio between the centimeter value and the pixel value.




```
Plant Height: 3369 pixels
Plant Height: 11.20 cm
```

We know the height of the plant above, and we get the height of the same plant in pixels, and based on that height, we get the ratio between the actual height of the plant and the pixel count. Using that, we calculate the height of the other plants.

These are approximate values, and we tried to always keep the distance between the plant and the camera uniform. Because depending on the distance between the camera and the plant, the pixel value captured by the camera on the plant changes.

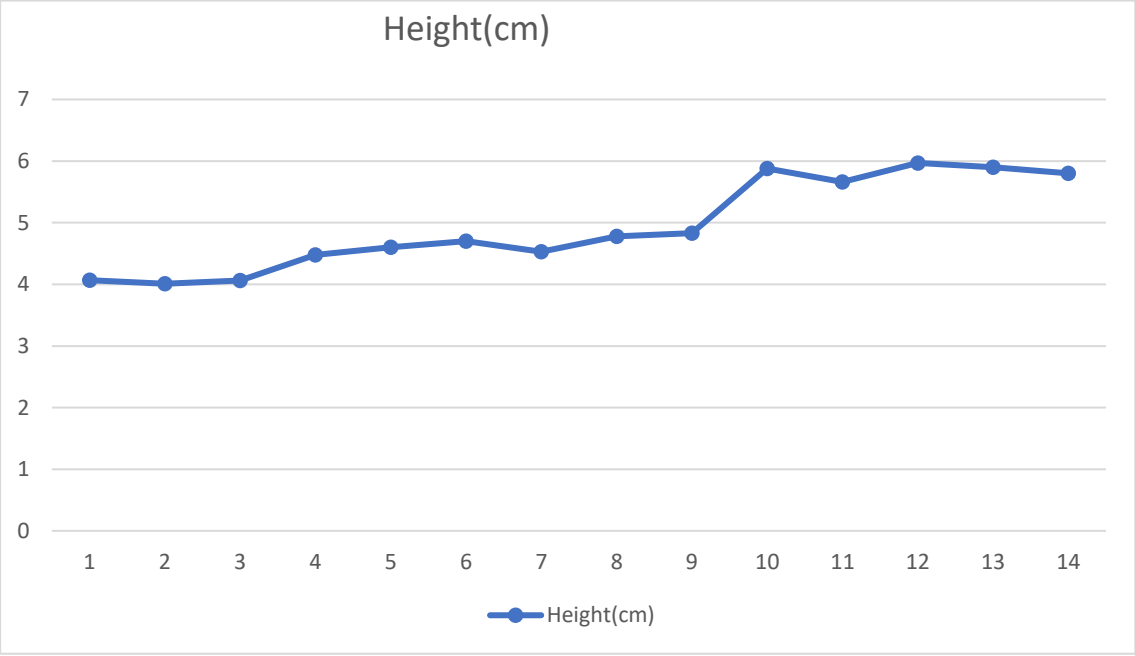

```
Plant Height: 1223 pixels
Plant Height: 4.07 cm
```


```
Plant Height: 1572 pixels
Plant Height: 5.23 cm
```

5.1.Calculated values for plant height detection

Day count	Pixel value	Centemetre value	rounding Values	rounding Values
1	1223	4.07	4.1	4
2	1207	4.01	4.0	4
3	1222	4.06	4.1	4
4	1349	4.48	4.5	5
5	1384	4.60	4.6	5
6	1413	4.70	4.7	5
7	1364	4.53	4.5	5
8	1437	4.78	4.7	5
9	1454	4.83	4.8	5
10	1769	5.88	4.9	5
11	1704	5.66	5.7	6
12	1795	5.97	5.9	6
13	1762	5.86	5.9	6
14	1733	5.76	5.8	6

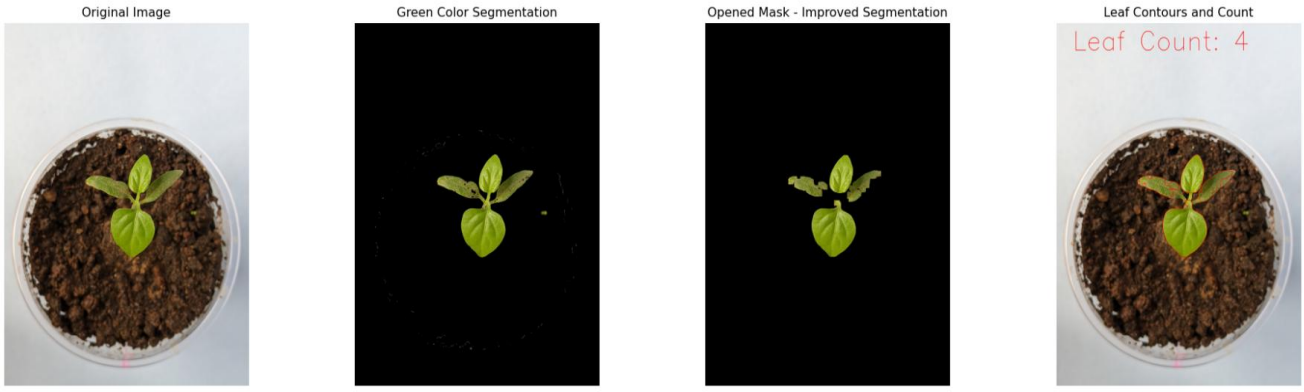
The measurements show that there is no uniformity between the number of days and the growth. However, this is not due to a difference in growth. As the plant turns towards the sun, the position of the leaves changes slightly, so there is a small discrepancy between these measurements. However, if we consider the measurement rounding , it seems that the growth of the plant has occurred gradually.



6. leaf count counting

After segmenting the plant region, we had to improve that code to count leaves. We improved the code as follows:

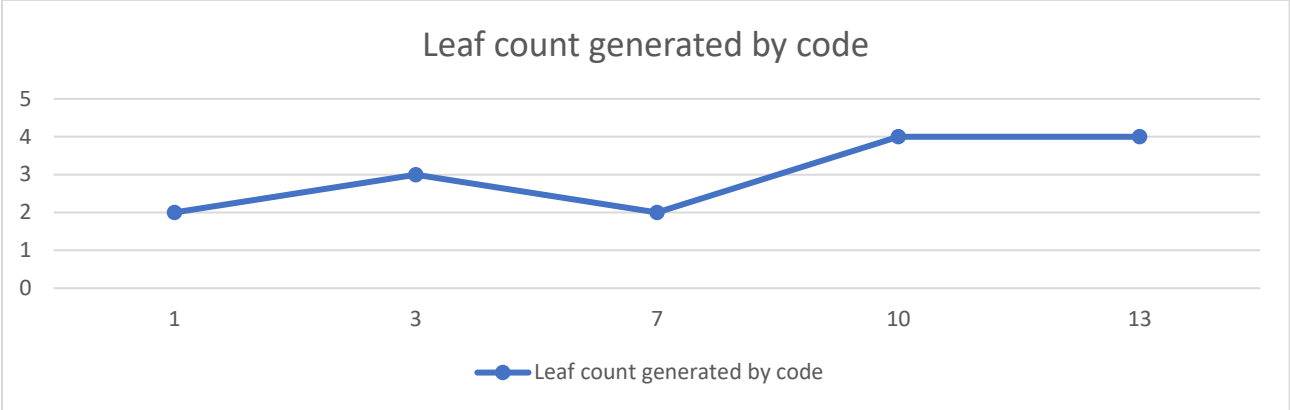
- Apply morphological opening to break narrow connections
    - To separate the leaves from trunk, We Changed kernel values for proper morphology
- Find contours of leaves
    - Count the separated pixel regions
- Filter contours based on area to remove small noise
    - Since the plants leave has a large number of pixel regions, we removed other small non-leaf regions.
- Draw contours on the original image and count them




```
Total Leaf Count: 4
```

6.1. Counting leaves in images and that data

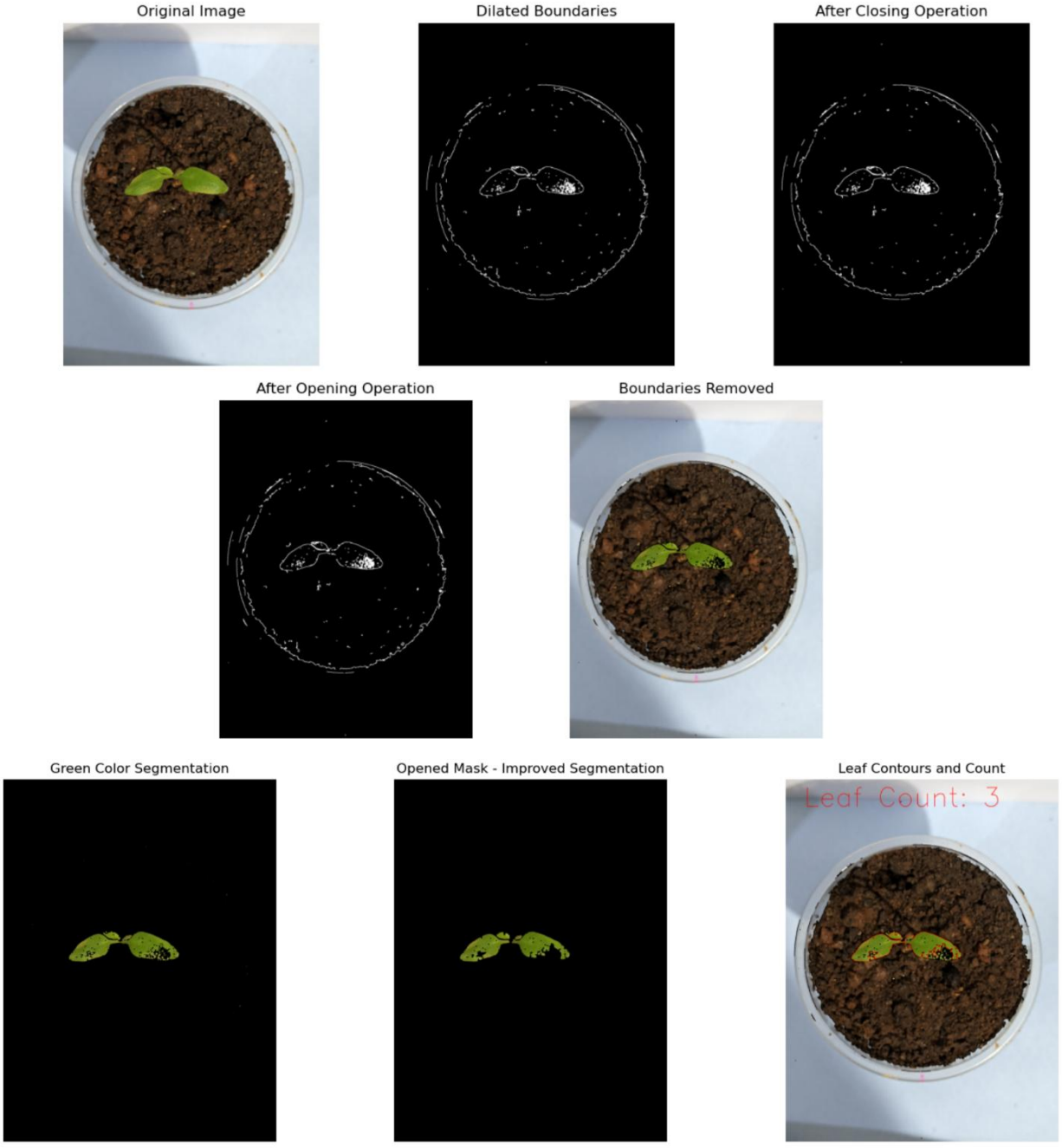
Day count	Actual Leaf count	Leaf count generated by code
1	3	2
3	3	3
7	4	2
10	4	4
13	4	4



The above figures show that the value derived from counting the number of leaves is not always match the its actual leaf count . The leaf count is not correct when the leaves overlap or Bound together. To avoided that mistake we further improved the code like bellow

- First we apply gaussian blur to image to improve accuracy of detecting edge
- Edge detection using Canny
- Dilate the edges to enhance boundaries
- Apply morphological closing (dilation followed by erosion) to strengthen edges

- Apply morphological opening (erosion followed by dilation) to remove noise
- Then we create a mask from that selected edge and invert mask
- After using that edge mask as boundary mask we Remove boundaries from the original image
- After we calculate Leaf count again



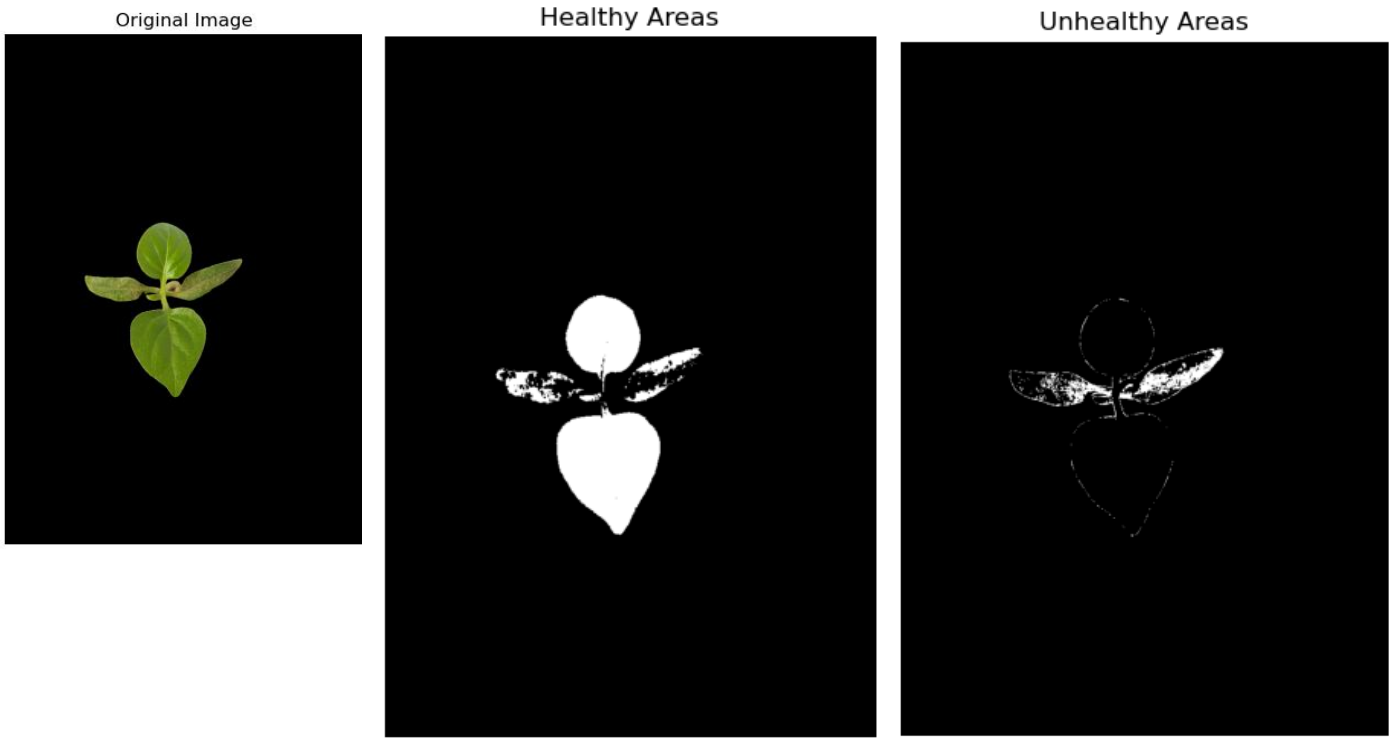
It was successful to some extent, but it wasn't always successful.

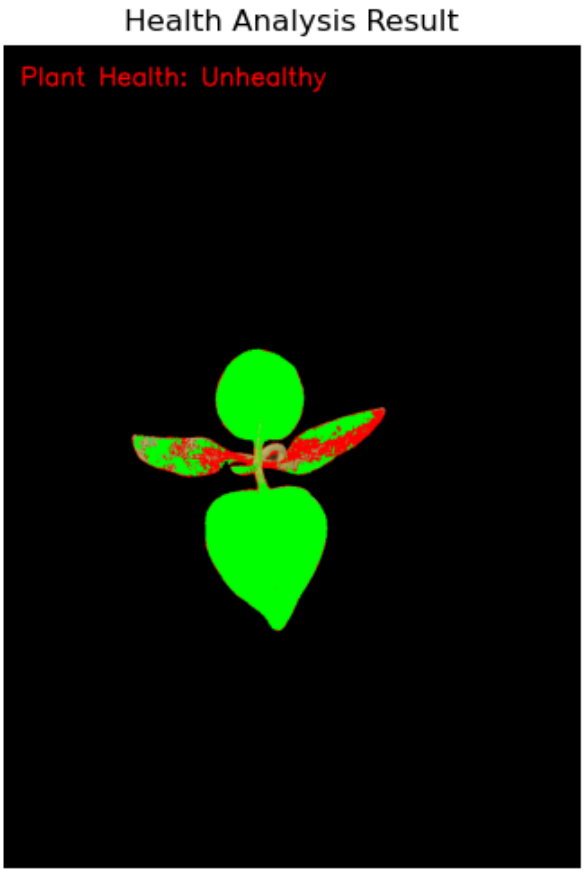
7. Identifying plant health

After segmenting the plant region, we had to improve that code to count leaves. We improved the code as follows:

- Healthy leaves are assumed to be predominantly green and are segmented using a specific HSV range.
- Unhealthy areas (yellow or brown) are segmented using another HSV range.
- Health Analysis
    - Healthy regions are highlighted in green and unhealthy regions in red on the original image.
- Output:
    - The program prints the plant is healthy or unhealthy.
    - It displays the original image alongside processed masks and a final annotated result.

7.1. Health Indication





- *Highlight healthy areas using Green color*
- *Highlight Unhealthy areas using Red color*
- *Define a simple heuristic for plant health based on color:*
    - *For example, if the average hue is in a typical healthy green range (45-75) and saturation is high (e.g., >100), then the plant is considered "Healthy."*

8. Reference

- https://www.youtube.com/watch?v=55eTVDto9K8&list=PLw7f_swC6ZDU5E4LN4bXXGpomHCIpGH4f
- https://docs.opencv.org/4.x/d7/d4d/tutorial_py_thresholding.html
- https://www.dataquest.io/blog/jupyter-notebook-tutorial/

10. Contribution

Name	CN No:	Index No:	Contribution
K.D.H.Rashmika	KR1211	5706	• Identify distortions in image • Remove gaussian noise • Edge sharpening to enhance plant details • Github handling • Plant height detection
R.D.M.Nethmini	MN1032	5689	• Reducing non-uniform illuminations • Health indication part
W.K.D.Bhagya	WB293	5713	• Create Noise reduction pipe line doc • Plant leaf counting
M.T.Rathnayake	MR1766	5716	• Remove salt and paper noise • Plant leaf counting (bounded leaves)